

Lazy Page Allocation in xv6

Project Documentation

Team Members

Gehad Almolla	231000894
Ohoud Khaled	231001119
Shadw Ali	231000705
Mariam Radwan	231001563
Areej Khaled	231001151

Problem Definition

In the original xv6 implementation, when a process requests memory using `sbrk()`, physical memory is allocated immediately. This is inefficient because a process may request large memory regions without actually using them.

The problem was to modify xv6 so memory allocation happens only when the process actually accesses the requested memory.

Objective

Implement **lazy page allocation (demand paging)** in xv6 by delaying physical memory allocation until a page fault occurs.

Expected Outcome

The modified system should:

- Increase process memory size without immediate allocation
- Allocate memory only on first access
- Handle page faults correctly
- Support normal execution after allocation
- Work correctly with `fork()` and process termination

Background

Lazy allocation is a memory management technique where memory is allocated only when needed rather than at request time.

This relies on:

- **Virtual Memory:** allows processes to use logical addresses
- **Page Tables:** map virtual addresses to physical memory
- **Page Faults:** triggered when an unmapped page is accessed

Modern operating systems use similar demand-paging mechanisms for efficiency.

Solution / Implementation Approach

1. Modify `sys_sbrk()` (`kernel/sysproc.c`)

The original `sbrk()` implementation called `growproc()` to allocate memory immediately.

Modification:

- For positive memory growth, only increase `p->sz`
- Do not allocate physical pages immediately

Result: The process reserves virtual address space only.

2. Handle Page Faults (`kernel/trap.c`)

When a process accesses an unmapped lazy page:

- detect page fault
- retrieve faulting address
- verify it belongs to valid process memory

- allocate a physical page using `kalloc()`
- clear it with `memset()`
- map it using `mappages()`
- resume execution

If allocation fails or access is invalid, the process is terminated.

3. Update Virtual Memory Handling (kernel/vm.c)

Some xv6 functions assume all pages exist.

Modified logic to safely handle missing lazy pages in:

- memory unmapping
- address space copying during `fork()`
- cleanup operations

This prevents kernel panics with sparse address spaces.

Testing Performed

To evaluate the correctness and stability of the lazy page allocation implementation, a dedicated test program (`lazymtest.c`) was developed and executed within xv6. The tests were designed to validate both the core functionality of lazy allocation and its compatibility with other operating system mechanisms.

1. Basic Lazy Allocation Test:

This test verified that memory requested through `sbrk()` is not allocated immediately, but only when first accessed. A single page of memory was requested, and data was written to and read from different locations within that page. The successful execution of this test

confirmed that page faults were correctly handled and that physical memory was allocated on demand.

2. Large Allocation Test:

To ensure scalability, a larger memory region spanning multiple pages was requested. Each page was then accessed individually to confirm that allocation occurred progressively as pages were touched rather than all at once. The test passed successfully, demonstrating stable lazy allocation behavior across multiple pages.

3. Fork Compatibility Test:

This test examined whether lazily allocated memory remained consistent when a process was duplicated using `fork()`. A parent process allocated memory, wrote data into it, and then created a child process. The child successfully accessed the same data, confirming that the address space copying mechanism correctly supported lazy allocation.

4. Process Size Verification Test:

A test using `sbrk(0)` was performed to verify that process memory size tracking remained accurate even though physical allocation was deferred. After requesting additional memory, the reported process size increased as expected, confirming that virtual memory expansion was handled correctly.

5. System Call String Access Test:

This test validated that system calls could correctly access strings stored in lazily allocated memory. A string was placed in a newly allocated lazy page and then passed as an argument to a system call. Successful execution confirmed that the kernel correctly handled lazy allocation during system call memory access.

lazyttest output:

```
hart 2 starting
hart 1 starting
init: starting sh
$ lazytest
=====
  Lazy Allocation Tests
=====

Test 1: Basic Lazy Allocation
mem[0] = A
mem[4095] = Z
Test 1 PASSED ✓

Test 2: Large Allocation
Test 2 PASSED ✓

Test 3: Fork with Lazy Memory
Child: memory correct ✓
Test 3 PASSED ✓

Test 4: sbrk(0) Size Check
Test 4 PASSED ✓

Test 5: Syscall String Access
Test 5 PASSED ✓

=====
  All Tests Done!
=====
```

Observed Results

The implementation worked successfully.

Results:

- memory is no longer allocated immediately with sbrk()
- page faults trigger dynamic allocation correctly
- processes continue execution normally after allocation
- sparse address spaces are handled safely
- system behavior is closer to real operating systems

Challenges Faced

Kernel assumptions: xv6 expected all pages to exist. Solution: updated VM functions to tolerate missing pages.

Fault validation: not every page fault should allocate memory. Solution: added address boundary checks.

Fork compatibility: child processes needed correct address-space handling. Solution: adjusted copy behavior.

Conclusion

This project successfully implemented lazy page allocation in xv6 by replacing eager allocation with demand-driven allocation triggered by page faults.

The result is a more memory-efficient and realistic operating system behavior while improving understanding of virtual memory and kernel memory management.